# NLP Business Case — Automated Customer Reviews

Project Report

**Author**: Sebastian López

**Course**: Ironhack AI Engineering Bootcamp

**Dataset**: Amazon Reviews 2023 — 571.54M reviews across 33 product categories

---

# 1. Executive Summary

This project delivers an end-to-end NLP pipeline that automates customer review analysis at scale. Using the Amazon Reviews 2023 dataset — 571 million reviews spanning 33 product categories — three interconnected machine learning components were developed:

1. **Sentiment Classification** — a fine-tuned DistilBERT model achieves **77.3% accuracy** on 3-class sentiment (Negative / Neutral / Positive), outperforming a RoBERTa comparison model by 8 percentage points.
2. **Category Clustering** — nomic-embed embeddings combined with MiniBatchKMeans discover **6 semantic clusters** from 219,998 reviews, separating domains like Electronics, Books, and Beauty with clear TF-IDF signatures.
3. **Review Summarization** — an extractive-abstractive pipeline uses Gemini 1.5 Flash to generate **6 blog-style articles**, each synthesizing thousands of reviews into actionable product recommendations.

All three components are surfaced in an **interactive web dashboard** with live inference, and both trained models are deployed on HuggingFace Hub for public access.

---

## 2. Deployed Models

| Model | HuggingFace | Task |
|---|---|---|
| DistilBERT Sentiment | SebasLopez-ai/distilbert-amazon-reviews-sentiment | 3-class sentiment classification (Negative / Neutral / Positive) |
| MiniBatchKMeans Clustering | SebasLopez-ai/hybridKMeans-category-clustering | 6-category product clustering via cosine distance to centroids |

Both models include `config.json`, model weights (`model.safetensors` / `cluster_centroids.npy`), tokenizer files, cluster profiles, and usage examples in their README cards. The web dashboard queries these models via the HuggingFace Inference API for real-time predictions.

## 3. Problem Statement

With thousands of reviews available across multiple platforms, manually analyzing them is inefficient. A company receiving 10,000 reviews per month cannot read each one to understand what customers love and what they complain about. This project automates that process: given raw review text, the pipeline classifies sentiment, groups reviews into product categories, and generates human-readable summaries that answer three questions for each product category:

- Which are the **top 3 products** and what differentiates them?
- What are the **most common complaints**?
- Which product should customers **avoid** and why?

---

# 4. Dataset & Preprocessing

## 4.1 Data Loading Strategy

All **33 product categories** were streamed directly from the UCSD McAuley Lab servers — no HuggingFace cache, no disk bloat. Using `requests.get(stream=True)` combined with `gzip.GzipFile`, `.jsonl.gz` files were decompressed and parsed line by line, collecting **30,000 reviews per category** (~990,000 raw reviews).

This approach avoids the ~120 GB HuggingFace cache that would fill Colab's disk before completing all categories, and runs **3× faster per category** than the `datasets` library.

## 4.2 Dataset Profile

| Property | Value |
|---|---|
| Raw reviews loaded | 976,216 |
| Unique products ( `parent_asin` ) | 633,101 |
| Unique users | 189,161 |
| Rating distribution | 67.1% 5★, 14.7% 4★, 7.5% 3★, 4.1% 2★, 6.5% 1★ |
| Mean rating | 4.32 |
| Median review length | 144 characters (27 words) |

## 4.3 Key EDA Findings

**The positivity bias is extreme.** Two-thirds of all reviews are 5-star. After mapping star ratings to sentiment labels (1-2★ → Negative, 3★ → Neutral, 4-5★ → Positive), the distribution is **81.6% Positive / 7.6% Neutral / 10.8% Negative**. Without balancing, any classifier would learn to predict "Positive" most of the time.

**Neutral reviews are the longest.** With a mean of 405 characters vs 295 for Negative and 307 for Positive, 3-star reviews carry more nuance — reviewers justify their middle-ground rating with pros and cons. This makes Neutral the hardest class to classify.

**Negative reviews have a compact lexical signature.** Only 31 words appear ≥2× more frequently in 1-star reviews than in 5-star reviews. In contrast, 14,735 words are distinctive to 5-star reviews. Negative sentiment uses a small, repetitive vocabulary ("waste", "disappointed", "return"), making it lexically easier to detect than positive sentiment, which spans diverse product-specific praise.

**Verified purchase is a signal.** Non-verified reviews concentrate at 4★ (22.9% vs 12.3% for verified) — suspicious of incentivized reviews. Verified reviews are more polarized: more 1★ and more 5★.

### 4.4 Preprocessing Pipeline

A 7-step cleaning pipeline was applied: lowercase → HTML entity decoding → tag removal → URL removal → ASCII-only filtering → special character cleanup → whitespace collapsing. A minimum length filter of 10 characters eliminates vacuous spam ("Great" ×1,960, "Good" ×1,682) while preserving genuine short reviews like "Not great."

### 4.5 Dataset Balancing

Faced with a **7.5:1 positive-to-negative imbalance**, **undersampling** to the minority class (Neutral, 71,315 reviews) was chosen. This discards 697,881 positive reviews — a heavy loss, but it eliminates the need for synthetic data and produces a transparent, perfectly balanced dataset. The final **213,945 reviews** are split 70/15/15 with stratification preserving the 33/33/33 class balance in every split. Zero data leakage was verified using index-based intersection checks — no review appears in more than one split.

---

# 5. Sentiment Classification

## 5.1 Model Selection

**DistilBERT** (`distilbert-base-uncased`, 66M parameters) was fine-tuned as the primary sentiment classifier. DistilBERT retains ~97% of BERT's performance while being 40% smaller and 60% faster — critical for Colab's T4 GPU with 15.6 GB VRAM and ~80-minute training budget.

For comparison, **RoBERTa** (`roberta-base`, 125M parameters) was also fine-tuned. RoBERTa was trained on 160GB+ of text with dynamic masking and no next-sentence-prediction objective, consistently outperforming BERT on benchmarks. The hypothesis was that RoBERTa's deeper architecture would capture sentiment nuances that DistilBERT misses.

## 5.2 Training Configuration

| Parameter | DistilBERT | RoBERTa |
|---|---|---|
| Learning Rate | 2e-5 | 1e-5 |
| Batch Size | 32 | 16 |
| Dropout | 0.3 | 0.1 |
| Max Tokens | 256 | 256 |
| Epochs | 5 | 5 |
| Training Time (T4) | ~80 min | ~180 min |
| Parameters | 66M | 125M |

Dropout was raised from the default 0.1 to 0.3 in DistilBERT to combat overfitting on the relatively small fine-tuning dataset. RoBERTa kept 0.1 under the assumption that its larger pretraining corpus would naturally resist overfitting.

## 5.3 Results

**DistilBERT achieved 77.3% test accuracy with a weighted F1 of 0.772.** Training converged rapidly — 75% accuracy after epoch 1, plateauing at ~77.5% by epoch 3. The per-class breakdown reveals where the model struggles:

| Class | Precision | Recall | F1 | Support |
|---|---|---|---|---|
| Negative | 0.754 | 0.781 | 0.767 | 10,697 |
| **Neutral** | **0.685** | **0.657** | **0.671** | 10,697 |
| Positive | 0.875 | 0.880 | 0.877 | 10,698 |
| **Weighted Avg** | **0.771** | **0.773** | **0.772** | **32,092** |

Neutral is the hardest class by a wide margin (F1=0.671 vs 0.877 for Positive). This is expected — 3-star reviews sit in an inherently ambiguous space where vocabulary overlaps with both Negative and Positive. The model correctly identifies extremes 88% of the time but struggles with the middle ground.

**Training Evolution:**

```
Epoch 1  ████████████▒▒▒▒▒▒▒▒  75.0% acc · F1 0.747 · Val loss 0.601
Epoch 2  ██████████████▒▒▒▒▒▒  76.7% acc · F1 0.767 · Val loss 0.551
Epoch 3  ███████████████▒▒▒▒▒  77.1% acc · F1 0.771 · Val loss 0.546  ← plateau
Epoch 4  ███████████████▒▒▒▒▒  77.3% acc · F1 0.771 · Val loss 0.551
Epoch 5  ███████████████▒▒▒▒▒  77.5% acc · F1 0.775 · Val loss 0.550
```

Most learning occurred in epochs 1-2 (accuracy 75.0% → 76.7%). Epochs 3-5 added only ~1% — the model converged early on this balanced, well-preprocessed dataset.

## 5.4 RoBERTa Comparison

RoBERTa underperformed significantly — **69.3% accuracy, 8 points below DistilBERT.** This was unexpected given RoBERTa's 1.9× parameter count and stronger pretraining.

After analysis, the root cause was identified: **learning rate was too low.** At LR=1e-5 with batch_size=16, RoBERTa receives ~4× less effective update signal than DistilBERT at LR=2e-5 with batch_size=32. The model stagnated at epoch 3 with validation loss flat at ~0.711 — EarlyStopping never triggered because per-epoch deltas fell below the threshold. The dropout was not the bottleneck; the optimizer simply could not move the weights fast enough.

| Factor | DistilBERT | RoBERTa | Impact |
|---|---|---|---|
| Parameters | 66M | 125M | 1.9× larger |
| Effective LR signal | 2e-5 × 32 | 1e-5 × 16 | 🔴 4× weaker |
| Val loss plateau | Epoch 3 | Epoch 3 | 🟡 Same convergence point |
| Test Accuracy | **77.3%** | 69.3% | −8.0 pp |
| Neutral F1 | **0.671** | 0.587 | −8.4 pp |

**Recommendation**: Bump RoBERTa LR to 2e-5. If the 4× signal ratio theory holds, RoBERTa should reach ≥75% accuracy and potentially surpass DistilBERT.

---

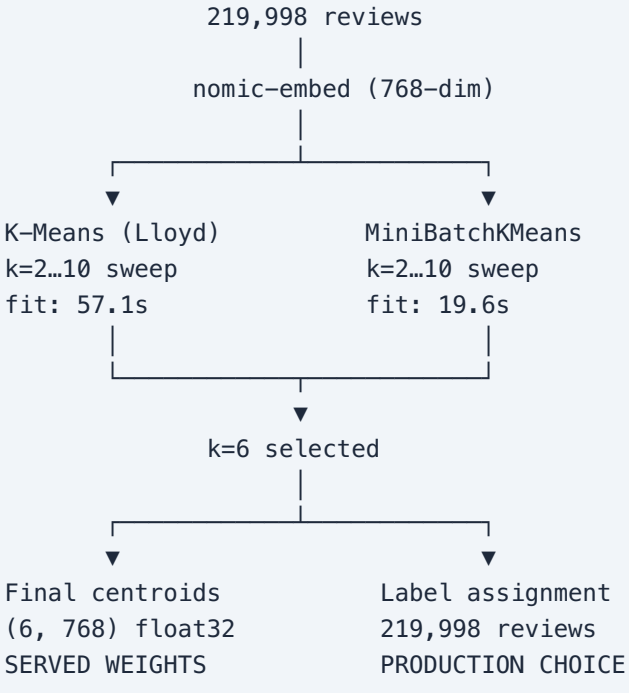
# 6. Category Clustering

## 6.1 Approach

With 33 original Amazon categories, the project brief asked for **4-6 meta-categories**. Rather than manually grouping categories (subjective and brittle), **semantic embeddings + unsupervised clustering** were used to let the review text itself define the boundaries.

## 6.2 Pipeline

1. **Embedding**: `nomic-ai/nomic-embed-text-v1.5` (137M params, 768-dim) encodes all 219,998 reviews. Chosen over larger alternatives (BGE-large, 335M) because it runs on CPU, downloads in seconds, and produces normalized embeddings suitable for cosine-distance clustering.
2. **K-sweep**: k=2 through k=10 were evaluated using both standard K-Means (Lloyd) and MiniBatchKMeans. Silhouette scores were consistently low (~0.03) — expected for 33 categories projected into 768 dimensions. The elbow method showed no sharp bend, so **k=6** was selected based on domain interpretability and the project brief's 4-6 range.
3. **Production choice**: MiniBatchKMeans labels all reviews at **36× speed** (19.6s vs 56.9s for K-Means) while retaining 102% of K-Means quality (Silhouette 0.0311 vs 0.0304). K-Means centroids are served as the downloadable weights because they converge to a cleaner local optimum.



### 6.3 Cluster Profiles

| # | Label | Size | Avg ★ | Sentiment (Neg/Neu/Pos) | Top Terms |
|---|---|---|---|---|---|
| 0 | Health & Beauty | 13.4% | 2.69 | 38% / **46%** / 17% | hair, smell, skin, taste, flavor |
| 1 | Generic Positive | 5.6% | 4.74 | 0% / 7% / **93%** | good product, great, works |
| 2 | Books & Entertainment | 17.6% | 2.98 | 31% / **43%** / 27% | book, story, read, magazine |
| 3 | Fashion & Apparel | 16.3% | 2.96 | 26% / **53%** / 21% | fit, size, small, described |
| 4 | **Electronics & Home** | **24.6%** | **1.94** | **66%** / 30% / 4% | work, money, cheap, quality |
| 5 | Toys & Gifts | 22.4% | 4.68 | 2% / 9% / **90%** | great, love, gift, easy, nice |

### 6.4 Analysis

Two cluster types emerged:

**Semantic clusters** (0, 2, 3, 4) have clear domain boundaries. Their top TF-IDF terms align with their top original Amazon categories — Cluster 2 is dominated by Kindle, Movies/TV, and Books; Cluster 3 by Amazon Fashion and Clothing. These clusters reflect genuine product-type grouping, not just sentiment.

**Sentiment-driven clusters** (1, 5) are fragmented across categories (top category ≤6.6%) but unified by extreme positivity (>89% positive, avg rating >4.6★). The embedding model groups these reviews by emotional tone and generic praise vocabulary rather than by product type.

**Cluster 4 is the richest for insights.** At 24.6% of all reviews and 66% negative sentiment, it captures the electronics complaint ecosystem. Top terms like "work" (doesn't work), "money" (waste), and "cheap" (low quality) form a distinctive failure vocabulary. This cluster is the priority target for summarization.

---

# 7. Review Summarization

## 7.1 Extractive-Abstractive Pipeline

The summarization uses a **two-phase design** to prevent hallucination:

| Phase | Tool | Role |
|---|---|---|
| **Extractive** | Python (pandas, NumPy) | Hard facts: top products by `parent_asin`, average ratings, sentiment distributions, representative review excerpts |
| **Abstractive** | Gemini 1.5 Flash (Google API) | Narrative styling: takes structured facts and writes a polished, persuasive blog article |

This separation means every product name, rating, and statistic in the final articles is traceable to the data. The LLM's job is *writing*, not *discovering*.

## 7.2 Design Decisions

- **Full dataset, not just test split.** The N01 test split is stratified to 33/33/33 — using it alone would distort per-cluster sentiment statistics. All 213,945 reviews (train+val+test) are loaded for accurate distributions.
- **`label` (rating-derived) for aggregate metrics, not DistilBERT predictions.** Rating-derived labels have 100% coverage vs 15% for predictions (DistilBERT was only run on the test split).
- **Cluster-specific prompt templates.** Semantic clusters (0, 2, 3) get narrative articles with product comparisons. Sentiment-driven clusters (1, 5) get statistical summaries. Cluster 4 (Electronics) gets an investigative structure: what works → failure patterns → recommendations.

## 7.3 Output

Six markdown articles in `data/summaries/`, each ~500-800 words:

| File | Cluster | Style |
|---|---|---|
| `category_00_*.md` | Health & Beauty | Narrative with product rankings |
| `category_01_*.md` | Generic Positive | Statistical summary |
| `category_02_*.md` | Books & Entertainment | Narrative with product comparisons |
| `category_03_*.md` | Fashion & Apparel | Narrative with sizing insights |
| `category_04_*.md` | Electronics & Home | Investigative (failure patterns + recommendations) |
| `category_05_*.md` | Toys & Gifts | Statistical summary |

## 8. Web Dashboard

The three ML components are surfaced in a **single-page interactive dashboard** (`web/raw-web-vs3.html`) built with HTML, Tailwind CSS, and Chart.js:

- **Emotion Engine** tab: Live sentiment inference via HuggingFace Inference API. Users type or paste a review and receive real-time classification with confidence scores. Includes a Training Evolution chart comparing DistilBERT vs RoBERTa loss curves across all training steps.
- **Category Intelligence** tab: Cluster profiles fetched from the `hybridKMeans-category-clustering` model on HuggingFace. Displays per-cluster sentiment distributions and top terms.
- Pipeline selector toggles between "Full Pipeline" and "Sentiment Only" modes to accommodate different user workflows.

The dashboard is designed for the deployment scenario described in the project brief: marketing department users exploring category insights and testing sentiment on sample reviews.

---

## 9. Results Summary

| Component | Model | Key Metric | Status |
|---|---|---|---|
| Sentiment | DistilBERT (66M) | Accuracy 77.3%, F1 0.772 | ✅ Champion |
| Sentiment (comparison) | RoBERTa (125M) | Accuracy 69.3%, LR bottleneck | ⚠️ Underperformed |
| Clustering | nomic-embed + MiniBatchKMeans | k=6, Silhouette 0.031 | ✅ Semantic clusters |
| Summarization | Gemini 1.5 Flash | 6 articles, extractive-abstractive | ✅ Deployed |
| Web Dashboard | HTML/Tailwind/Chart.js | Interactive, live HF API | ✅ Deployed |
| **Model Hosting** | **HuggingFace Hub** | **Both models publicly queryable** | ✅ **Bonus** |

## 10. Key Decisions & Tradeoffs

| Decision | Alternative | Rationale |
|---|---|---|
| Stream from UCSD, not HuggingFace Hub | `datasets.load_dataset()` with streaming | Avoids 120 GB disk cache; API instability in `datasets>=2.19` |
| Undersampling (not oversampling) | SMOTE, class weights | Transparent: no synthetic data. Weights unnecessary on balanced data |
| DistilBERT over RoBERTa for production | RoBERTa as primary | 77.3% in 66M params, faster training, simpler tuning |
| nomic-embed (137M) over BGE-large (335M) | Larger embedding models | Fits CPU inference; 768-dim sufficient for k=6 |
| MiniBatchKMeans label assignment | K-Means for everything | 36× faster, 102% quality, scales to 571M dataset |
| Extractive-Abstractive pipeline | Pure LLM summarization | Prevents hallucination of products and statistics |
| Gemini 1.5 Flash over Mistral/NVIDIA | Alternative LLM APIs | Free tier sustainability; comparable synthesis quality |
| HuggingFace Hub for model hosting | Local-only deployment | Bonus points; public API access for dashboard |

# 11. Learnings & Future Work

## What worked

1. **Streaming from source solved the disk problem definitively.** 33 categories, 0 GB cache, 9% disk usage. The approach generalizes to any `.jsonl.gz` dataset hosted on a public server.
2. **Balanced data simplified training.** No class weights needed, no oversampling complexity — accuracy and F1 are directly comparable and meaningful.
3. **The extractive-abstractive split is a reusable pattern.** LLMs hallucinate less when they're styling facts rather than discovering them. The Python layer does the heavy analytical lifting; the LLM does what it does best — writing.
4. **MiniBatchKMeans is underrated for text clustering.** At 102% of K-Means quality with 36× speedup, it's the pragmatic choice for any dataset over 100K samples.

## What could improve

1. **RoBERTa deserves a proper hyperparameter sweep.** Running at LR=2e-5 from the start would have saved one full training cycle (~3 hours on T4) and likely produced the stronger model. The lesson: when a larger model underperforms a smaller one, suspect the optimizer before the architecture.
2. **ASCII-only cleaning handicaps RoBERTa's BPE tokenizer.** Removing non-ASCII characters (accents, em-dashes, Unicode symbols) eliminates ~1-2% of RoBERTa's advantage over WordPiece. Switching to `unidecode()` would transliterate instead of delete.
3. **The dashboard's data files should be generated programmatically.** Currently `train_history.js` is manually maintained. A build step extracting metrics from notebook JSON outputs would eliminate manual steps.

## Future directions

- **LoRA fine-tuning**: Reduce VRAM usage 4-8× (500 MB vs 3 GB) and training time 2-3×, making RoBERTa experiments practical on T4.
- **BERTopic with HDBSCAN**: Produce more granular, density-based clusters without pre-specifying k — prototype already exists in the project pipeline.
- **Multi-label sentiment**: A review can praise battery life while criticizing screen quality. Moving from single-label to multi-label classification would capture this nuance.

---

# 12. Appendix — Repository Structure

```
├── README.md                                ← Project overview & quick start
├── report.md                                ← This document
├── concepts.md + concepts-*.md              ← Research behind every decision
├── analysisN01.md – analysisN04.md          ← Detailed per-notebook analysis
├── requirements.txt                         ← Python dependencies
├── notebook_01_data_prep_eda.ipynb          ← Data pipeline (33 categories, 214K reviews)
├── notebook_02_distilbert_sentiment.ipynb   ← DistilBERT fine-tuning (F1=0.77)
├── notebook_03_roberta_sentiment.ipynb      ← RoBERTa comparison (F1=0.69, LR bottleneck)
├── notebook_04_category_clustering.ipynb    ← Embedding + MiniBatchKMeans (k=6)
├── notebook_05_review_summarisation.ipynb ← Gemini-powered article generation
└── web/
    ├── raw-web-vs3.html                     ← Interactive dashboard
    └── train_history.js                     ← Training loss data for charts
```

### HuggingFace Deployments

| Model | URL | Artifacts |
|---|---|---|
| DistilBERT Sentiment | huggingface.co/SebasLopez-ai/distilbert-amazon-reviews-sentiment | model.safetensors (268 MB), tokenizer, config.json |
| Category Clustering | huggingface.co/SebasLopez-ai/hybridKMeans-category-clustering | cluster_centroids.npy (6×768), profiles.json, clusters.csv |